

COSC 612/AIT 624

Introduction

Dr. Akshita Maradapu Vera Venkata Sai
Department of Computer and Information Sciences
Towson University

Outline

1. Professional Software Engineering

What is meant by software engineering

2. Software engineering ethics

Different ethical issues

3. Case Studies

Introduces 4 examples that will be used in later chapters

What is Software Engineering ?

software engineering $\neq$ programming

- Problem Solving

- Understand the problem
- Propose a solution and a plan
- Engineer a system for the proposed solution using good design

- Dealing with complexity

- Creating different abstractions and models

- Knowledge Management

- Elicitation, analysis, design, validation of the system and the solution process

What is Software Engineering ?

Software engineering is a collection of **techniques, methodologies** and **tools** that help with :

- the production of a high quality software
- developed within a budget
- before a given deadline
- while change occurs

CHALLENGE : Dealing with complexity and change

What is Software Engineering ?

Software engineering is an engineering discipline that is concerned with all aspects of software production from initial conception to operation and maintenance

Two key phrases :

- **Engineering discipline**
Using appropriate **theories** and **methods** to solve problems bearing in mind organizational and financial constraints.
- **All aspects** of software production
 - Not just technical process of development.
 - Also **project management** and **the development of tools, methods etc. to support software production**.

Why Software Engineering ?

- The economies of all developed nations are dependent on software.
- More and more systems are software controlled
- Expenditure on software represents a significant fraction of GNP in all developed countries.
- We need to be able to produce reliable and trustworthy systems economically and quickly.
- Software costs often dominate computer system costs.
- We need cost-effective software development

Why is Software Engineering Important?

- Reliable and trustworthy systems need to be produced economically and quickly to meet the demand for new software systems
- In the long run using software engineering methods is cheaper

Factors for Software Failure

Increasing system complexity :

- As new software engineering techniques help us to
 - build **larger**, more **complex** systems, the demands change.
- Systems have to be built and delivered more quickly;
 - larger, even more complex systems are required (**Amazon ecommerce, Google search engine**);
 - systems have to have new capabilities that were previously thought to be impossible.

Factors for Software Failure

Failure to use software engineering methods

- It is fairly easy to write computer programs **without using software engineering methods and techniques.**
- Many companies have drifted into software development as their products and services have evolved.
 - They do not use software engineering methods in their everyday work.
 - Consequently, their software is often more expensive and less reliable than it should be.

Professional Software Engineering

Software engineering

What is Software?

- Computer Programs and associated documents
- Developed for a particular customer or for a general market

Difference between software engineering and computer science?

Computer Scientist

- Assumes **techniques** and **tools** have to be developed.
- Proves theorems about algorithms, designs languages, defines knowledge representation schemes
- Has infinite time...

Software Engineer

- Works in multiple application domains
- Has only 3 months...
- ...while changes occurs in the problem formulation (requirements) and also in the available technology

Software engineering

What is the difference between software engineering and system engineering?

- System engineering is concerned with all aspects of computer-based systems development including **hardware, software** and **process** engineering.
- Software engineering is part of this more general process.

What are the key challenges we face in software engineering?

- Coping with increasing **diversity**,
- demands for **reduced delivery times**
- and developing **trustworthy software**.

What are the costs of software engineering?

- Roughly **60%** of software costs are development costs, **40%** are testing costs.
- For **custom** software, evolution costs often **exceed development costs**.

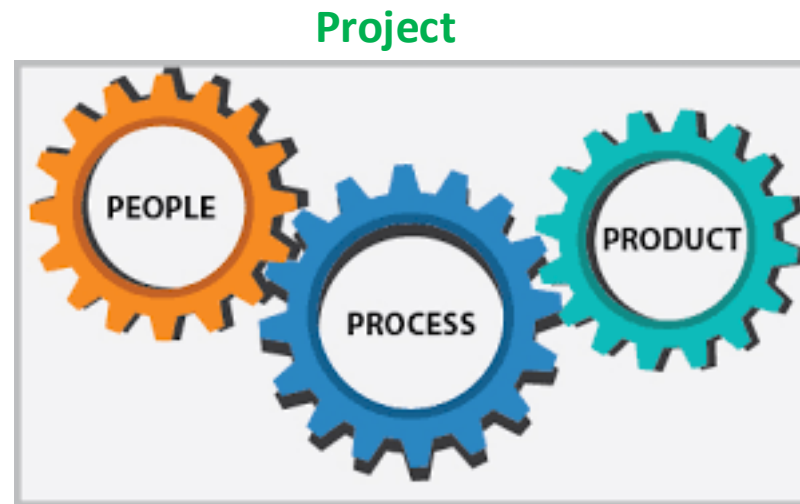
Software engineering

What are the best software engineering methods and techniques?

- Different techniques for different systems
- For example:
 - Games – developed using a series of prototypes
 - Safety critical control operations - require a **complete** and **analyzable** specification to be developed.
 - You can't, therefore, say that one method is better than another

4 P's of Software Engineering

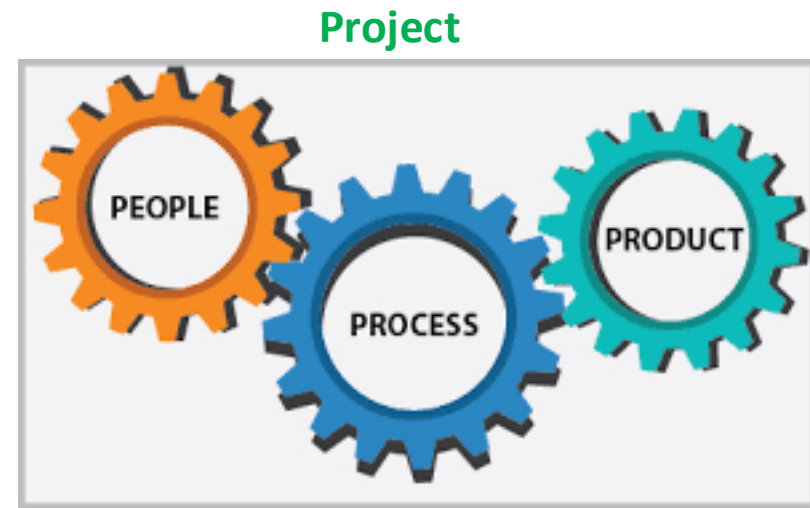
4 Ps of Software Engineering : People, Product, Process and Project



4 P's of Software Engineering

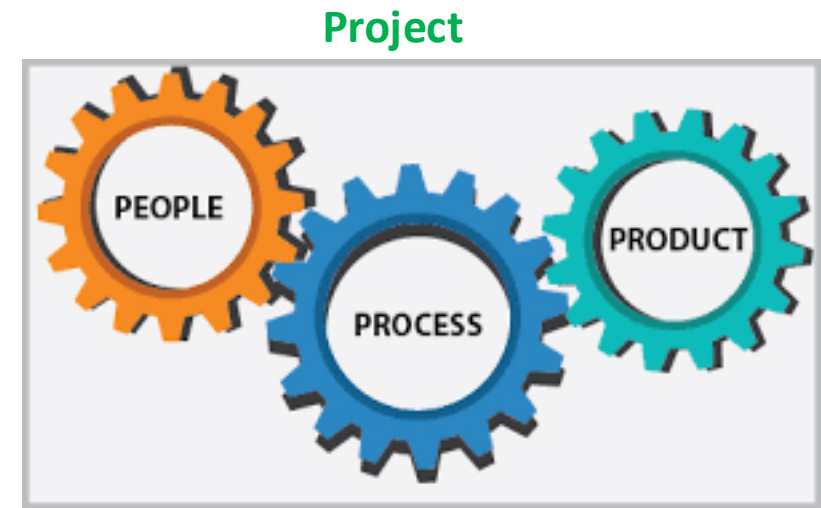
1. People

- Communication
- Collaboration
- Cooperation



4 P's of Software Engineering

- Role of people in software:
 - **customer** / client: wants software built
 - **often doesn't know what he/she wants**
 - **managers**: make plans, coordinate team
 - **hard to foresee all problems in advance**
 - **developers**: design and write code
 - **hard to write complex code for large systems**
 - **testers**: perform quality assurance (QA)
 - **impossible to test every combination of actions**
 - **users**: purchase and use software product
 - **can be fickle and can misunderstand the product**



4 P's of Software Engineering

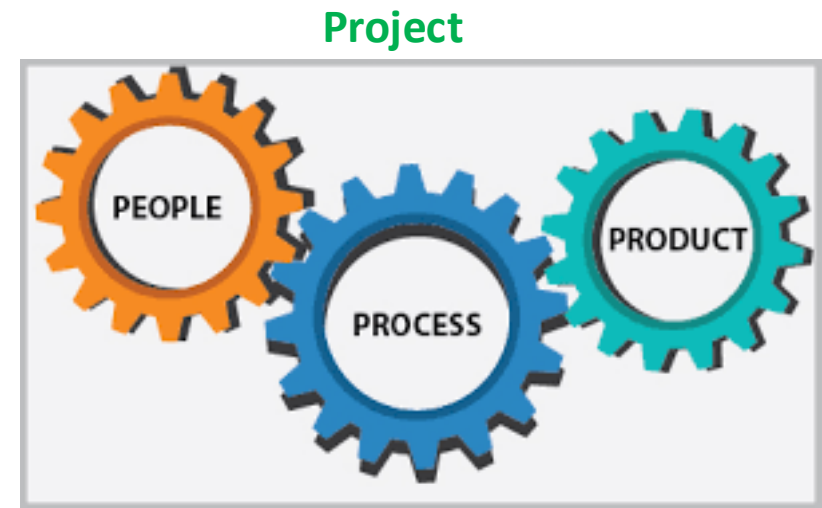
2. Product

Generic Products

- Stand alone systems produced by development organizations
- Sold on open market to any customer who is able to buy them
- Ex : apps, PC software, project management tools

Customized products

- Commissioned by and developed for a particular customer
- Ex : Air traffic control software



4 P's of Software Engineering

2. Product

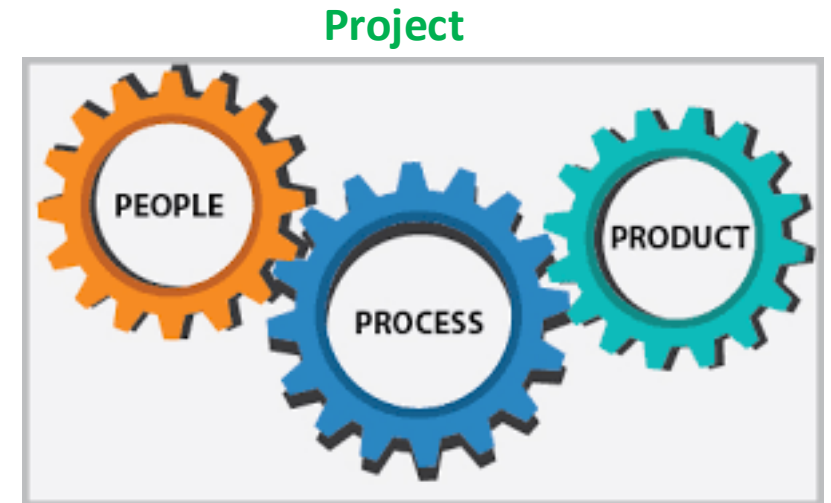
Attributes of a good software :

Acceptability:

- Acceptable to the type of users it is designed for
- Understandable, usable and compatible with the devices they use

Dependability and Security :

- Should not cause physical and economic damage in case of system failure
- Secure - to avoid malicious users from accessing the system



4 P's of Software Engineering

2. Product

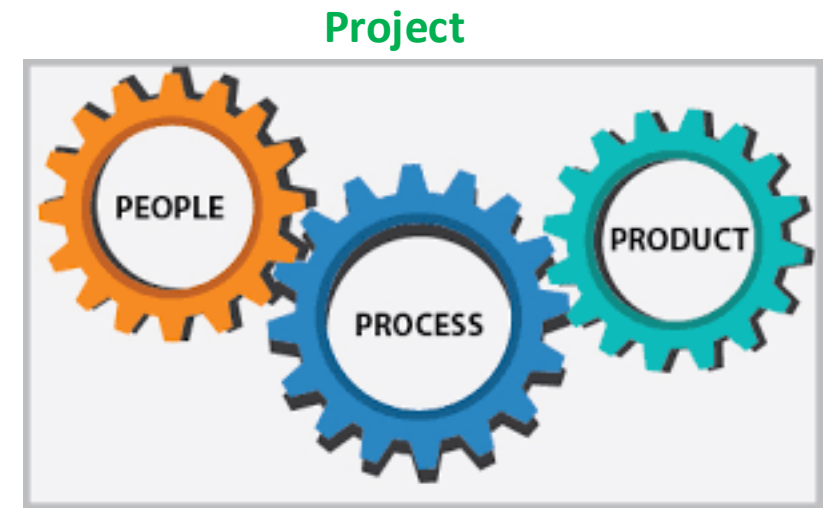
Attributes of a good software

Efficiency

- Should not make wasteful use of resources like memory
- It includes responsiveness, processing time, resource utilization

Maintainability

- Should be able to evolve to meet the changing needs of customers

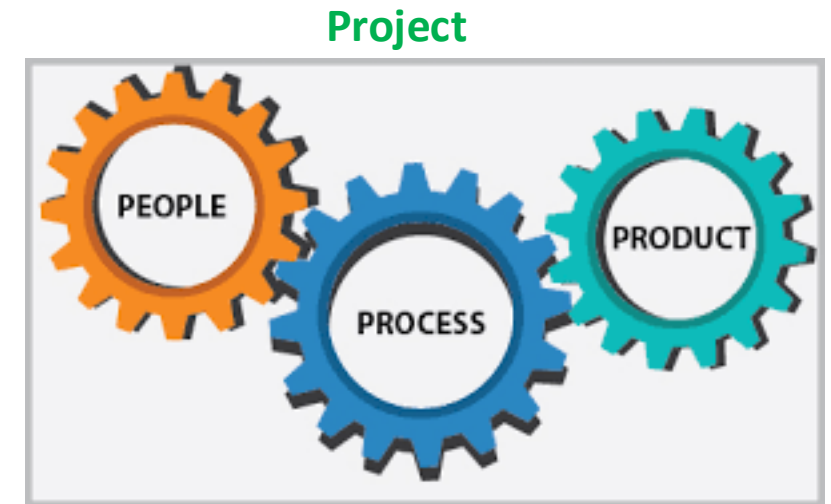


4 P's of Software Engineering

3. Process

Four fundamental activities of a software product:

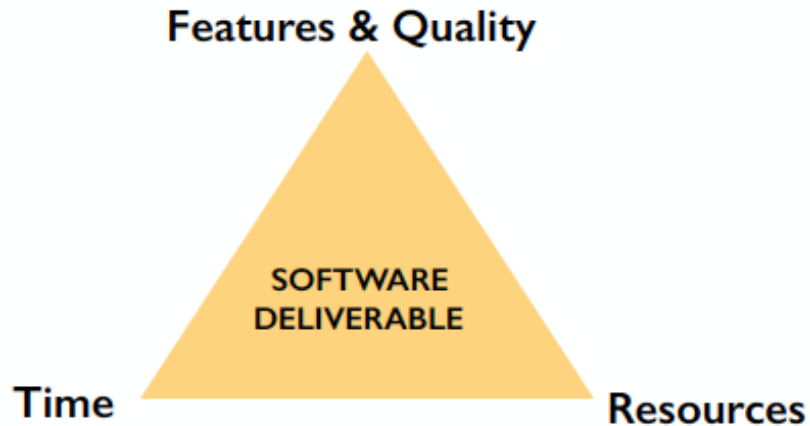
- **Software specification**, where **customers and engineers define** the software that is to be produced and the **constraints** on its operation.
- **Software development**, where the software is **designed** and **programmed**.
- **Software validation**, where the software is **checked** to ensure that it is what the customer requires.
- **Software evolution**, where the software is **modified** to reflect changing customer and market requirements.



4 P's of Software Engineering

4. Project

Project is a balance of three dimensions, with the goal of **producing a successful deliverable**.



Project development stages

- Proposal
- Requirements
- Design
- Implementation
- Testing, validation, verification
- Documentation
- Customer exposure
- Final deliverable

Project



Diversity in Software Engineering

There are many software systems in the market

- No one set of rules or techniques to follow for all of them

The software techniques and tools to be used depends on

- Type of the application to be developed
- Requirements of the customer
- Background of the development team

Diversity in Software Engineering

Types of Applications :

- Stand-alone applications
- Interactive transaction-based applications
- Embedded control systems
- Batch processing systems
- Entertainment systems
- Modelling and Simulations
- System of Systems

Fundamentals of Software Engineering

Fundamentals that apply to all types of software :

- Systems should be developed using a **managed and understood** development **process**.
 - Of course, different processes are used for different types of software.
- **Dependability** and **performance** are important for all types of system.
- Understanding and managing the **software specification and requirements** are important.
- Where appropriate, you should **reuse software** that has already been developed rather than write new software.

Issues in Software

Heterogeneity

- Systems operate on a distributed network
- Includes different kinds of computer and mobile devices
- The software has to execute on a variety of devices
- Challenge : develop software that is dependable and can cope with this heterogeneity

Business and Social Change

- Technology is evolving and new technology becomes available
- The software developed should be in way that it reduces the time for updates

Issues in Software

Security and Trust

- As software is intertwined with all aspects of our lives,
- it is essential that we can trust that software.

Scale

- Software has to be **developed across a very wide range of scales**,
 - from very small embedded systems in portable or wearable devices
 - through to **Internet-scale, cloud-based systems** that serve a global community.

Software Engineering Ethics

Issues of Professional Responsibility

Confidentiality

- Engineers should respect the confidentiality of the clients
- irrespective of whether or not they signed a confidential agreement

Competence

- **Engineers** should not misrepresent their level of competence.
- They should not knowingly accept work which is outwith their competence.

Issues of Professional Responsibility

Intellectual property rights

- **Engineers** should be aware of local laws governing the use of intellectual property such as patents, copyright, etc.
- They should be careful to ensure that the intellectual property of employers and clients is protected.

Computer misuse

- **Software engineers** should not use their technical skills to misuse other people's computers.
- Computer misuse ranges from
 - relatively trivial (game playing on an employer's machine, say)
 - to extremely serious (dissemination of viruses).

Case Studies

- **A personal insulin pump**

- An embedded system in an insulin pump used by diabetics to maintain blood glucose control.

- **A mental health case patient management system**

- Mentcare. A system used to maintain records of people receiving care for mental health problems.

- **A wilderness weather station**

- A data collection system that collects data about weather conditions in remote areas.

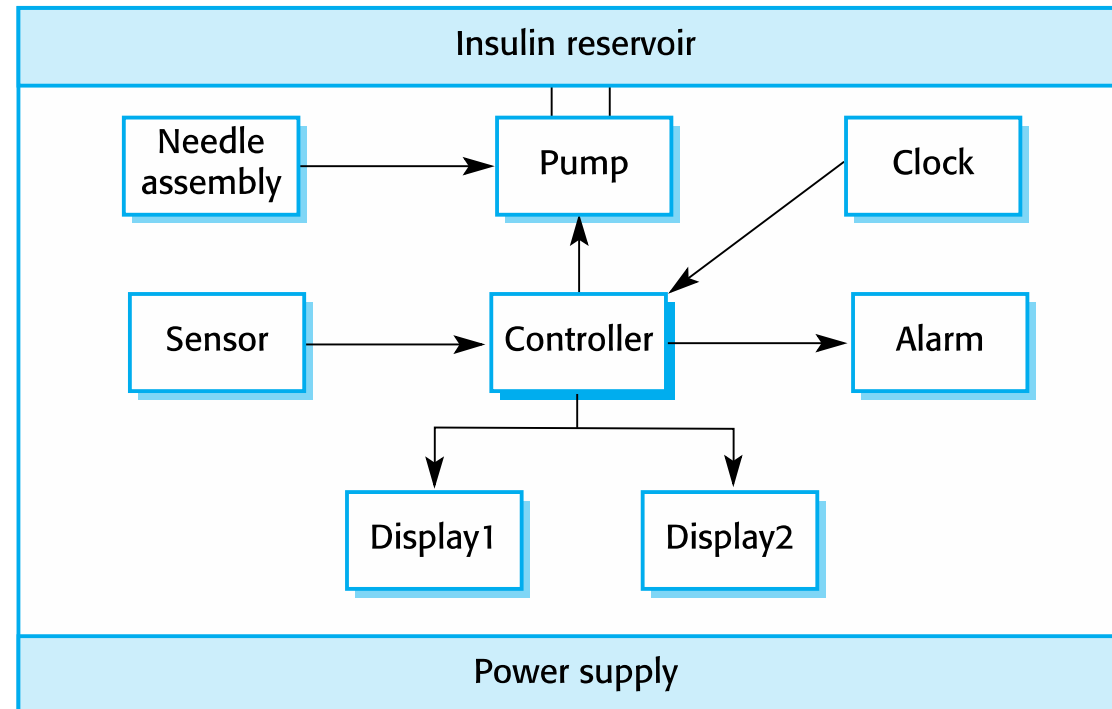
- **iLearn: a digital learning environment**

- A system to support learning in schools

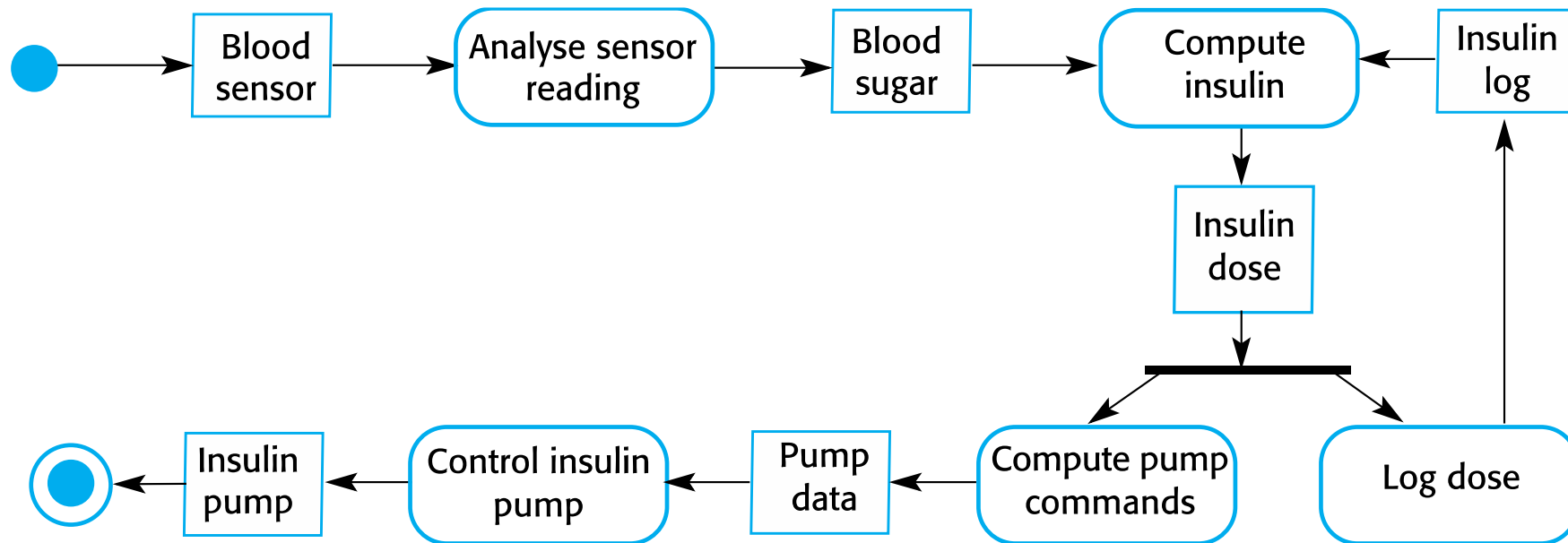
Insulin Pump Control System

- **Collects data from a blood sugar sensor and**
 - calculates the **amount of insulin required to be injected**.
- **Calculation** based on the rate of change of blood sugar levels.
- **Sends signals to a micro-pump to deliver the correct dose of insulin.**
- **Safety-critical system**
 - as low blood sugars can lead to brain malfunctioning,
 - coma and death;
 - high-blood sugar levels have long-term consequences such as eye and kidney damage.

Insulin Pump Hardware Architecture



Insulin Pump Activity Model



Essential High-level Requirements

- The system shall be
 - available to deliver insulin when required.
- The system shall
 - perform reliably and deliver the correct amount of insulin
 - to counteract the current level of blood sugar.
- The system must therefore be
 - designed and implemented to ensure that the system always meets these requirements.

Reading Assignment

- Go over the all the other case studies in this chapter

Key points

- Software engineering is **an engineering discipline** that is concerned with all aspects of software production.
- Essential software product attributes are
 - maintainability,
 - dependability and security,
 - efficiency and acceptability.
- The high-level activities of **specification, development, validation and evolution**
 - are part of all software processes.
- The fundamental notions of software engineering are
 - universally applicable to all types of system development.

Key points

- There are many different types of system each
 - requires appropriate software engineering tools and techniques for their development.
- The fundamental ideas of software engineering are
 - applicable to all types of software system.
- Software engineers have responsibilities to the engineering profession and society.
 - They should not simply be concerned with technical issues.
- Professional societies publish codes of conduct which set out the standards of behaviour expected of their members.